Runbook: checkout-api

When to use this runbook

Open this runbook when either of these alerts fires for checkout-api:

- `checkout-api-p99-latency-high` (p99 > 1500ms for 5 min)
- `checkout-api-error-rate-high` (5xx+4xx rate > 2%)

Which alert fired determines which triage path below applies — they lead to different investigations, not the same checklist.

Triage: if `p99-latency-high` fired

1. **Check the connection pool dashboard.**

Panel: `checkout-api > Postgres Pool > active_connections / max_connections`

- If `active_connections` is pinned at `max_connections` -> go to Known Issue #1 (connection pool exhaustion).

2. **Check cache hit rate.**

Panel: `checkout-api > Redis > cache_hit_ratio`

- If it has dropped sharply (e.g. ~95% -> under 50%) -> go to Known Issue #2 (cache node failover).

3. **Check downstream dependency latency.**

Panel: `checkout-api > Downstream Latency (by dependency)`

checkout-api calls `payment-service`, `inventory-svc`, and `fraud-scoring-svc` synchronously — a slow call to any of these shows up in checkout-api's own latency even though checkout-api itself is healthy.

- If a dependency's p99 spiked at the same time as checkout-api's -> this is not a checkout-api incident. Escalate to that dependency's on-call instead of continuing to investigate here — there is no local fix.

4. If none of the above resolve it, escalate per the path below.

Triage: if `error-rate-high` fired without elevated p99 latency

Fast failures point at the request or the response contract, not resource

contention — a different failure family from Known Issue #1/#2 below, which are both fundamentally slowness-driven.

1. **Break down the error rate by status code.**
Grafana panel: `checkout-api > Errors by status code (4xx vs 5xx)`

2. **If 4xx is what's climbing:**

- **400 Bad Request** -> the payload doesn't match the expected schema (missing field, wrong type, invalid JSON). Check for a recent deploy that changed the request contract (see `code-docs/checkout-api-request-schema.md` for the current expected schema), or a client that hasn't updated to match it. Check whether the errors are concentrated on one endpoint or one caller — that points at a specific client sending requests that no longer match what checkout-api expects, rather than a checkout-api-side bug.

- **401 Unauthorized** -> auth token missing, expired, or invalid. Check for a recent token-rotation or auth-service change, and whether the errors are concentrated on one client/token rather than spread evenly across all callers.

3. **If 5xx is what's climbing** (and the pool/cache checks in the other branch are clean):

- Check application error logs for exception stack traces — this points at a code-level bug rather than a resource-exhaustion pattern.

- Check whether the errors are concentrated right after a recent deploy — this is a genuine signal here, unlike the latency branch, because a fast-failing bug is exactly what a bad code change looks like.

Known Issue #1: Postgres connection pool exhaustion

Symptom signature: p99 latency climbs gradually (not a sudden step) as `active_connections` rises from baseline toward `max_connections`, pins at the ceiling while pressure holds, then eases back down as pressure subsides. Application logs show `could not obtain connection from pool within <N>ms`.

Mechanism: each pod holds a fixed pool of pre-opened database connections (cheaper than opening one per request). When every connection is checked out and a new request arrives, it waits for one to free up. Latency rises while requests wait; once the wait exceeds the pool's acquire timeout, waiting requests start failing outright — which is why latency climbs before the error rate does.

Contributing factors vary — a recent deploy touching pool/timeout config is one possible lead, but so is an organic traffic increase or an autoscaling mismatch between pod count and the database's connection ceiling. Don't assume a deploy is the cause without checking; treat the deploy timeline (triage step above) as one independent lead among several.

Immediate mitigation (no deploy required):

- Increase the PgBouncer pool size to give the service more headroom while the underlying cause is investigated:

1. Edit `infra/pgbouncer/checkout-pool.ini`, raise `default_pool_size` (e.g. from 20 to 35) and/or `max_client_conn` as needed.
2. Reload PgBouncer without dropping existing connections:

```
psql -h <pgbouncer-host> -p 6432 pgbouncer -c "RELOAD;"
```

3. Confirm the new size took effect via the admin console:

```
psql -h <pgbouncer-host> -p 6432 pgbouncer -c "SHOW POOLS;"
```

- Before increasing, check headroom against Postgres's own `max_connections` ceiling (currently 200) — raising PgBouncer's pool size only helps if Postgres itself has room above current demand. If PgBouncer's new ceiling would push total connections close to or above 200, this shifts the bottleneck to Postgres rather than relieving it.
- This raises the ceiling, it doesn't fix the underlying cause — treat it as headroom to stabilize the service while triage continues, not a resolution.
- Monitor `active_connections` for 5 minutes after the change before taking further action.

Known Issue #2: Redis cache node failover

Symptom signature: a sudden (not gradual) step-change in latency,

`cache_hit_ratio` drop, `CLUSTERDOWN` or `MOVED` errors in application logs.

Mitigation:

- Confirm failover completed: `redis-cli -c -h checkout-cache.internal cluster info` should show `cluster_state:ok`.
- If `cluster_state` is not `ok`, page the infra on-call (`#infra-oncall`) — this is outside checkout-api team's remediation scope.
- If `cluster_state` is `ok` but hit ratio stays low, the cache is likely still warming up; this typically self-resolves within 10-15 minutes as traffic repopulates it. No action needed beyond monitoring.

Escalation path

- L1 (0-15 min): checkout-api on-call, follow this runbook.
- L2 (15-30 min, unresolved): page payments-platform team lead, open incident channel `#inc-checkout-latency`.
- L3 (30+ min, or customer-facing revenue impact): declare SEV1, page the incident commander rotation.

Resolution criteria

- p99 latency back under 400ms for 10 consecutive minutes.
- Error rate back under 0.1%.
- No further connection-pool, cache, or error-rate alerts for 15 minutes.